



Data-Driven Security Analysis and Defensive Response

CSC3106 Cyber Security Fundamentals Mini-Project Report

Group : BB

Group Member	Glasgow ID	SIT ID	Contribution
Habib Noor Bin Shahul Hameedib	3070624B	2401040	25%
Yale Tan	3070645T	2401316	25%
Wei Feng Ryson Toh	3070647T	2401333	25%
Nabil Bin Noor Azlan	3070549B	2402978	25%

Part 1: Data-Driven Authentication Log Analysis.....	3
1. Security question and framing.....	3
2. Python-verified findings.....	3
3. Interpretation.....	4
4. Risk summary.....	5
5. Initial response recommendations.....	5
6. Assumptions, uncertainty and limitations.....	5
Part 2: Technical Defensive Response.....	6
1. Risk addressed and why it is the right priority.....	6
2. Proposed technical response.....	6
3. Does it work: testing against the assigned extract.....	7
4. Limitations, trade-offs and residual risk.....	8
Appendix A: Asset-Focused Risk Matrix.....	9
Appendix B: AI-Use Declaration.....	11
Appendix C: Group Workload Record.....	13
Appendix D: Supplementary Figure.....	14
References.....	15

Part 1: Data-Driven Authentication Log Analysis

1. Security question and framing

This analysis addresses one question: which authentication events or patterns in Group BB's extract of web01's auth.log (12,000 lines, 6–12 July) warrant priority for investigation or response, and why? All quantitative claims below were verified against the assigned extract using our analysis script, with parsing decisions documented in the accompanying README.

2. Python-verified findings

The extract contains 3,724 failed password attempts, 1,991 accepted logins, 1,468 invalid-user notices, and 443 "maximum authentication attempts exceeded" errors.

Four source IPs, 203.0.113.77, 198.51.100.8, 203.0.113.101, and 198.51.100.140, account for 2,555 of the 3,724 failed attempts (69%), well above any other IP in the extract (Figure 1).

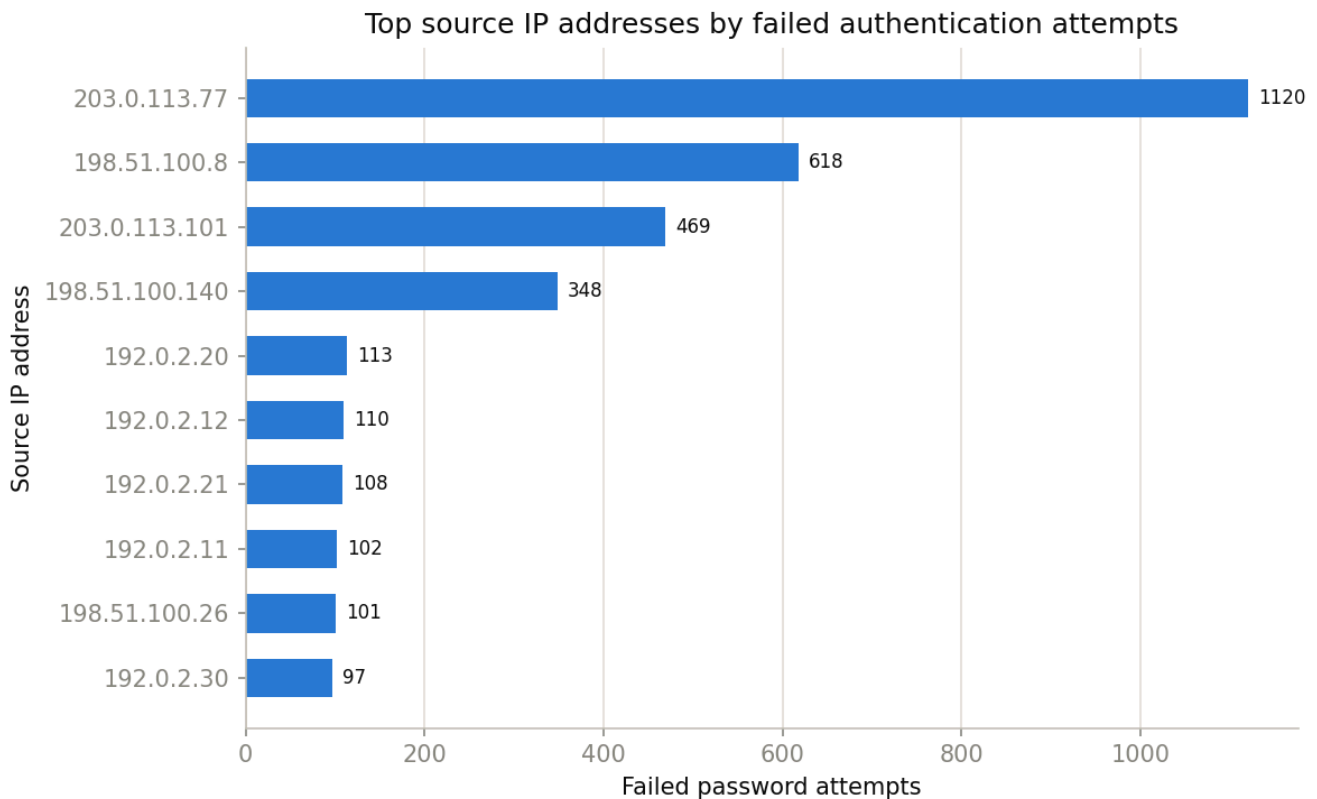


Figure 1. Top source IP addresses by failed authentication attempts (required visualisation).

These IPs concentrate on privileged accounts: deploy (428 failed attempts), webadmin (340) and ops (212) are the three most-targeted usernames in the extract, ahead of ordinary accounts such as alice (204) (Figure 2). root was targeted 149 times, but no successful root login appears anywhere in the extract.

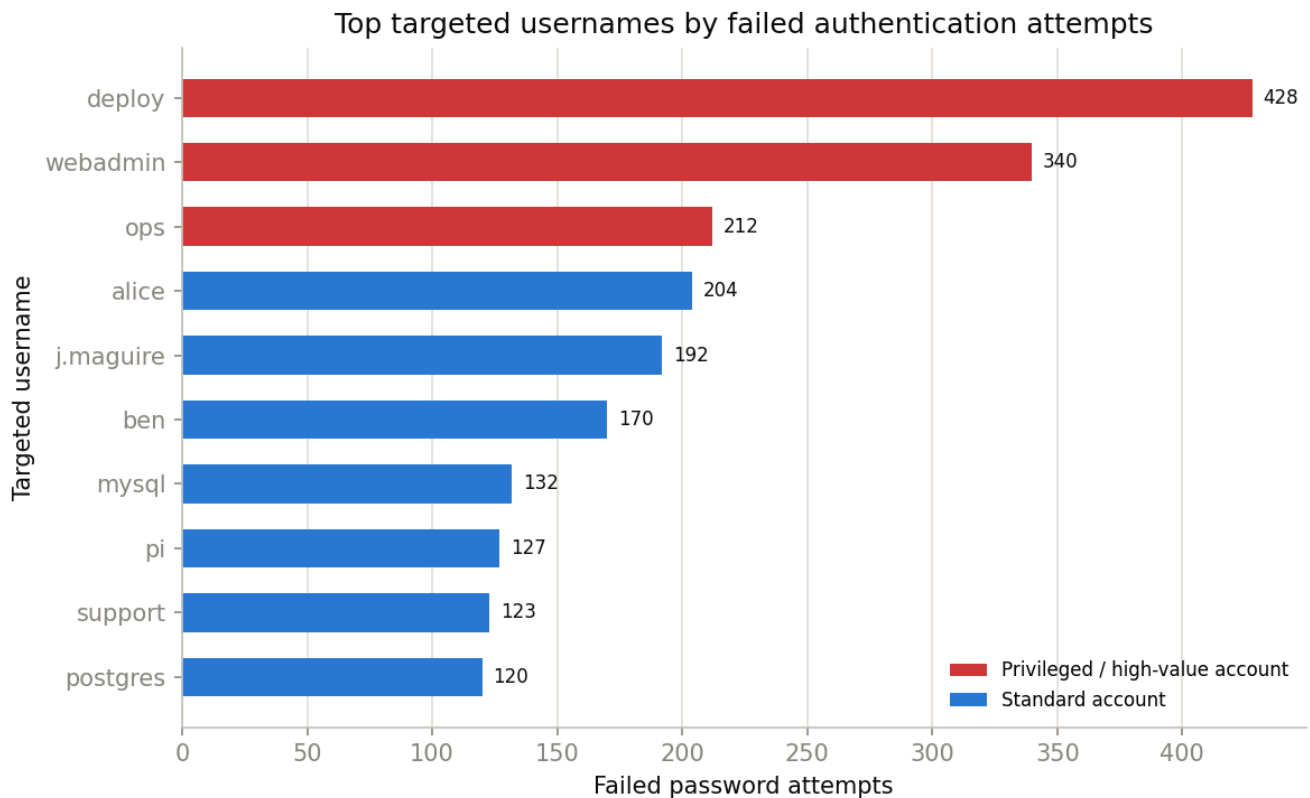


Figure 2. Top targeted usernames, privileged accounts highlighted (chosen second visualisation).

Activity is bursty, not constant. Failed attempts spiked on 7 July, with 1,161 for the day, about 49% more than the next-busiest day (779 on the 8th). The quietest day, 9 July, has only 165 (Appendix D, Figure 3).

The most significant event is a successful brute-force attack. From 23:13 to 23:47 on 11 July, the IP address 203.0.113.77 sent 117 uninterrupted failed password attempts against the deploy account. At 23:47:04, one of these attempts succeeded: 'Accepted password for deploy'. Within seconds, the same session ran 'sudo systemctl restart apache2' as root.

3. Interpretation

Together, this points to an automated credential-guessing campaign that prefers privileged accounts and has already succeeded once. The command run after compromise is ordinary in isolation: it recurs 60 times elsewhere in the extract from accounts with no reason for suspicion, so it is the authentication pattern, not the command itself, that identifies this session as anomalous. web01 currently has no way to distinguish that from routine admin activity on its own.

4. Risk summary

In priority order (full ratings and evidence in Appendix A):

- The deploy account compromise: confirmed, and deploy holds sudo/root-equivalent access.
- The ongoing credential-guessing campaign from the four flagged IPs.
- The absence of a working SSH rate limit or lockout: the condition that let the compromise happen.
- Direct guessing against other privileged accounts, not yet successful but high impact if it works.
- No way to distinguish attacker sessions from admin sessions once a login has succeeded.

5. Initial response recommendations

- Treat deploy as compromised now: rotate its credential immediately and review actions taken in and after the 23:47:04 session.
- Block or throttle the four flagged IPs pending investigation.
- Add rate-limiting or lockout to SSH password authentication (developed in Part 2).
- Require MFA or key-based authentication for root, deploy, webadmin, ops and sysadmin.
- Alert on ‘burst of failures immediately followed by a success’, the pattern that would have caught this in real time.

6. Assumptions, uncertainty and limitations

This is one host over one week; the pattern may not generalise to other systems, to web01 outside this window, or to other groups' extracts of the same base dataset. The source IPs sit in RFC 5737 documentation ranges, consistent with a sanitised teaching dataset, so no attribution or geolocation is attempted. The log alone cannot prove the deploy login was illegitimate, only that the burst-then-success pattern is strong circumstantial evidence; a stricter reviewer might call it probable rather than confirmed. No firewall, IDS, or file-integrity log is available in this extract, so whether some control already existed and was bypassed cannot be determined from auth.log alone.

Part 2: Technical Defensive Response

1. Risk addressed and why it is the right priority

This response builds on Risk 3 from Appendix A: SSH password authentication on web01 has no working rate limit or lockout. It is the enabling condition for Risks 1 and 2: the deploy compromise (Risk 1) only happened because one IP was allowed 117 uninterrupted guesses in 34 minutes, and the ongoing campaign (Risk 2) persists because nothing pushes back on failed attempts. It also carries the strongest log evidence of any Part 1 finding: 117 unbroken failures, and 443 "maximum authentication attempts exceeded" errors that the campaign simply reconnected past. Unlike Risk 1, whose remediation is largely procedural (credential rotation, session audit), Risk 3 is addressable through configuration and code, which is what Part 2 asks for.

2. Proposed technical response

- **Layer 1**, Prevent, hardens the SSH service itself. A fail2ban jail (jail.local) bans an IP for one hour, doubling on each repeat ban, after 5 failed attempts in 10 minutes. Alongside it, an sshd hardening configuration (50-hardening.conf) sets PermitRootLogin no (root has zero legitimate logins in the extract), MaxAuthTries 3, LoginGraceTime 30, and a MaxStartups limit against pre-auth connection flooding. A commented-out phase two moves the four privileged accounts to key-only authentication once keys are issued.
- **Layer 2**, Detect, is implemented in detector.py: three rules tuned against this extract's own legitimate-traffic baseline. R1 (critical) flags a success immediately after a burst of 10 or more failures from one IP, exactly the deploy incident. R2 flags any IP reaching 25 failures in one burst (legitimate IPs never exceed 9; campaign bursts never fall below 348). R3 flags any privileged account accumulating 15 failures in an hour regardless of source, so it still fires if an attacker rotates IPs to evade Layer 1 and R2.
- **Layer 3**, Preserve, forwards auth/authpriv logging to a separate collector (90-forward-auth.conf), so a compromised web01 session with sudo access cannot quietly edit the evidence it generated.

3. Does it work: testing against the assigned extract

The full extract was replayed against the policy. It bans exactly the four campaign IPs and nobody else; no legitimate login is ever blocked, since the worst legitimate burst measured is 9 failures spread over more than 10 minutes. 2,510 of 3,724 failed attempts (67%) never reach sshd (Figure 4).

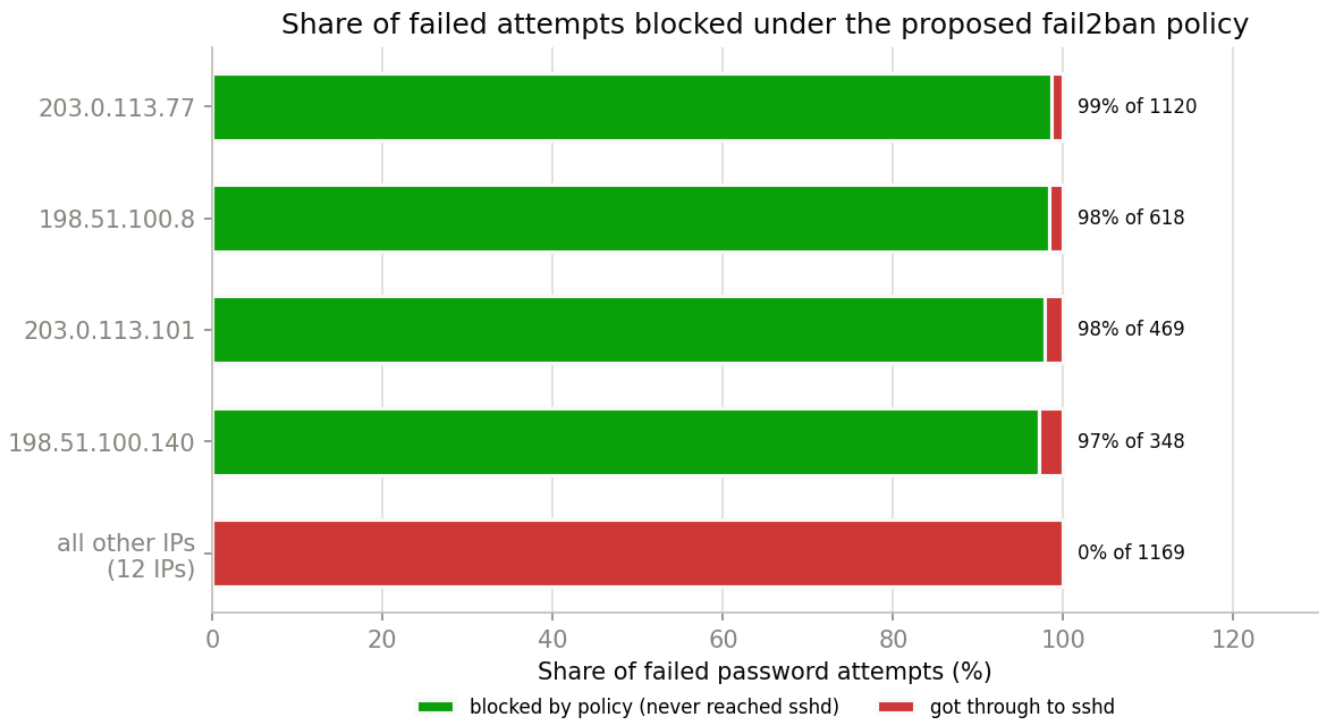


Figure 4. Failed attempts blocked under the proposed fail2ban policy, by source IP.

Critically, 203.0.113.77's ban at 23:13:45 (its third of the week) covers 23:47:04, the actual compromise time: under this policy, the Part 1 incident does not happen. The detection layer raises 10 alerts across the week, all genuine campaign activity, a triage load of one to two per day. In the final burst, R2 fires at 23:19:41, 27 minutes before the compromise (Figure 5), and had already flagged the same IP four days earlier (7 July, 02:05).

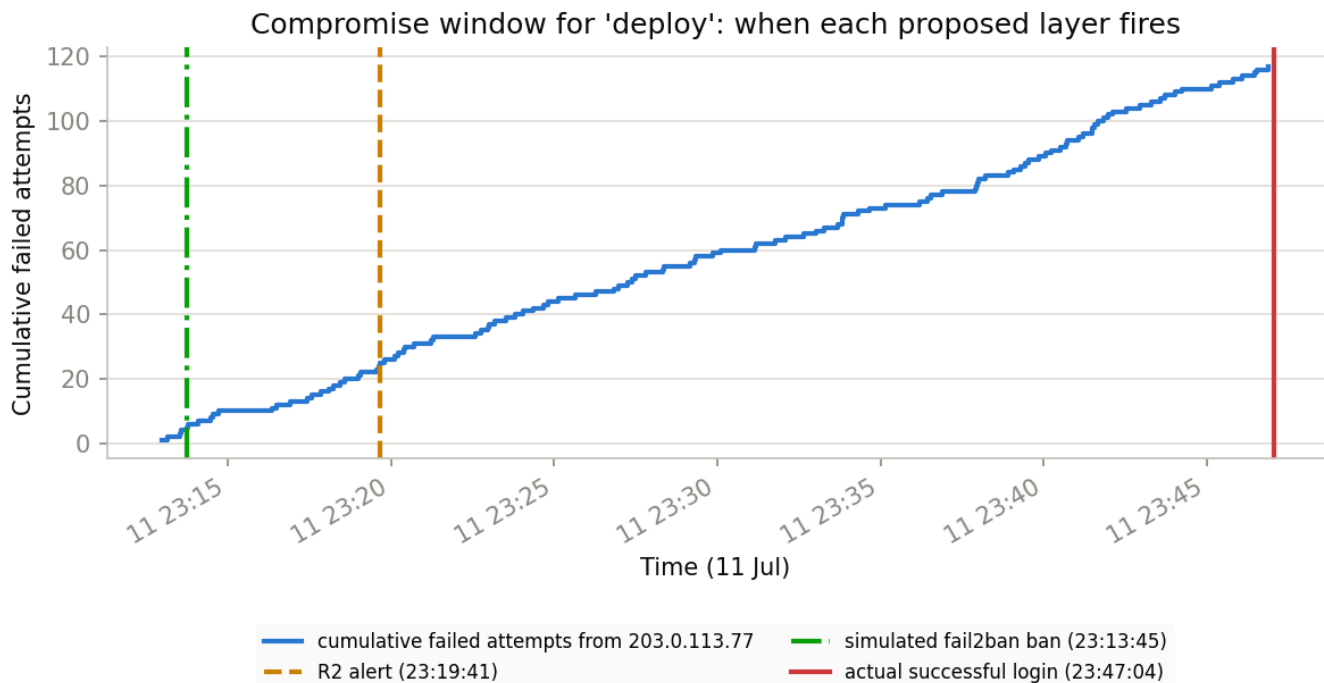


Figure 5. Detection and prevention timeline for the 11 July compromise window.

In risk-matrix terms: Layer 1 removes the observed likelihood of Risk 3's consequence; Layer 2 cuts detection time from days of manual review to under seven minutes; Layer 3 protects the investigability of anything that still gets through; and the phase-two key migration removes password guessing against the four most-targeted accounts entirely.

4. Limitations, trade-offs and residual risk

- The simulation replays recorded traffic; a real attacker who sees bans landing can rotate IPs or slow below the threshold. R3 (keyed on target account, not source) covers IP rotation; slow guessing is harder to detect but takes proportionally longer.
- Lockout is itself a denial-of-service surface if an attacker can spoof or share an admin's IP; jail.local ships with ignoreip left blank rather than guessed.
- Moving to key-only authentication has a real operational cost: deploy, webadmin and ops authenticate by password 330 or more times each this week, so PasswordAuthentication no ships commented out until keys are issued and tested.
- All thresholds are tuned to one week of one host and exposed as command-line flags so they can be re-derived on other logs.
- Layer 3 protects the log after it is written, not events sshd never logs. Residual risk beyond authentication, what an attacker does with a session once logged in, needs host-level controls such as auditd and is out of scope for this response.

Appendix A: Asset-Focused Risk Matrix

Asset: web01, the host the extract came from, reachable over SSH, including the privileged and service accounts on it (root, deploy, webadmin, ops, sysadmin) that can sudo to root and change system state.

Why it matters: web01 serves the organisation's web application and is administered remotely by a small set of named accounts over SSH. Unauthorised access to any of those accounts is not just a web01 problem, it is a problem for whatever web01 runs.

#	Risk / finding	Confidence	Likelihood	Impact	Why rated this way	What remains uncertain
1	web01 was accessed by someone who is not the legitimate owner of the deploy account	Confirmed	High	Critical	117 failed passwords from 203.0.113.77 in a 34-minute window, then a successful login, then a sudo command run as root within seconds. deploy has sudo-to-root rights. Likelihood is High rather than only Confirmed because the credential is still live and Risk 3 was still open at the end of the extract. .	Only one command is visible from that session; auth.log cannot show whether files or other accounts were touched.
2	Ongoing, concentrated credential-guessing against web01 from a handful of external IPs	High	High	High	Four IPs account for 2,555 of 3,724 failed attempts (69%); two tripped sshd's own break-in warning 35 times combined. Still active at the end of the extract.	Cannot tell whether this is one actor rotating addresses or several unrelated scanners.
3	SSH password authentication on web01 has no working rate limit or lockout	High	High	High	117 uninterrupted failures from one IP in 34 minutes; 443 "maximum authentication attempts exceeded" errors show sshd's per-connection cap firing while the attacker simply reconnected. A structural gap that applies to every future attempt until fixed.	No firewall or fail2ban log is present in this extract, so it is unclear whether a control exists and is failing, or nothing is there at all.

4	Privileged and service accounts on web01, not just deploy, are being directly guessed	Medium	Medium	Critical if successful	root was targeted 149 times (109 failed, 40 invalid-user); deploy, webadmin and ops are the three most-targeted usernames overall. No accepted root login appears in the extract.	Only deploy is known to have been compromised; whether the others have been guessed successfully outside this week cannot be answered from this log.
5	Once logged into web01, there is little to distinguish attacker activity from ordinary admin activity	High	High	Medium to High	The one command run in the compromised session, systemctl restart apache2, is identical to 60 other legitimate uses in the extract. A structural blind spot that recurs on every login.	It is unknown whether the compromised session did anything beyond the one visible command.

Appendix B: AI-Use Declaration

Tool used: Claude (Anthropic), via the web chat at claude.ai, for both Part 1 and Part 2.

What it was used for:

- Interpreting the auth.log format: message types, [preauth], and the distinction between "Failed password for X" and "Failed password for invalid user X"
- Drafting analysis.py, detector.py, simulate_lockout.py, and authlog_parsing.py
- Drafting the fail2ban, sshd, and rsyslog configuration files
- Suggesting the check for failures immediately followed by a success, the check that surfaced the deploy compromise

How outputs were checked: Every figure in this report came from running analysis.py (Part 1) or detector.py / simulate_lockout.py (Part 2) against the assigned extract and reading the output files, not from asking a model to summarise the log. This caught two errors during review:

- An early brute-force count of 1,120 failures turned out to be summed across five separate days, since the counter never reset. Corrected to the true value: 117 failures in one 34-minute window.
- A draft figure of "2,935 of 3,724 failed attempts (79%)" for the four flagged IPs didn't match top_source_ips.csv. Corrected to 2,555 (69%).

Which parts were substantially AI-shaped: Claude generated the initial version of the brute-force detection logic, which counted failed attempts per source IP with no reset condition as this is what produced the 1,120 figure, silently summing across five separate days instead of finding a single attack window. We rewrote it to group attempts into gap-based bursts (BURST_GAP_MINUTES) before flagging a streak, which is what surfaced the real 117-attempt, 34-minute finding. We also rewrote the timestamp-parsing error handling after it failed on malformed lines in our actual dataset.

What was decided by the group:

- Choosing web01 as the asset
- Selecting and ordering the five risks
- Choosing Risk 3 as the Part 2 focus
- Tuning every Part 2 threshold against the group's own measured baseline rather than accepted defaults
- Running the lockout simulation rather than asserting the policy would work
- Verifying every output file line by line before it entered the report

Reproducing the claims: Every number in this report traces to a file in part1/output/ or part2/output/, generated by running:

```
python part1/analysis.py 1_auth_log.txt --outdir part1/output
python part2/detector.py 1_auth_log.txt --outdir part2/output --plot
python part2/simulate_lockout.py 1_auth_log.txt --outdir part2/output
```


Appendix C: Group Workload Record

All workload was split equally by all members.

Appendix D: Supplementary Figure

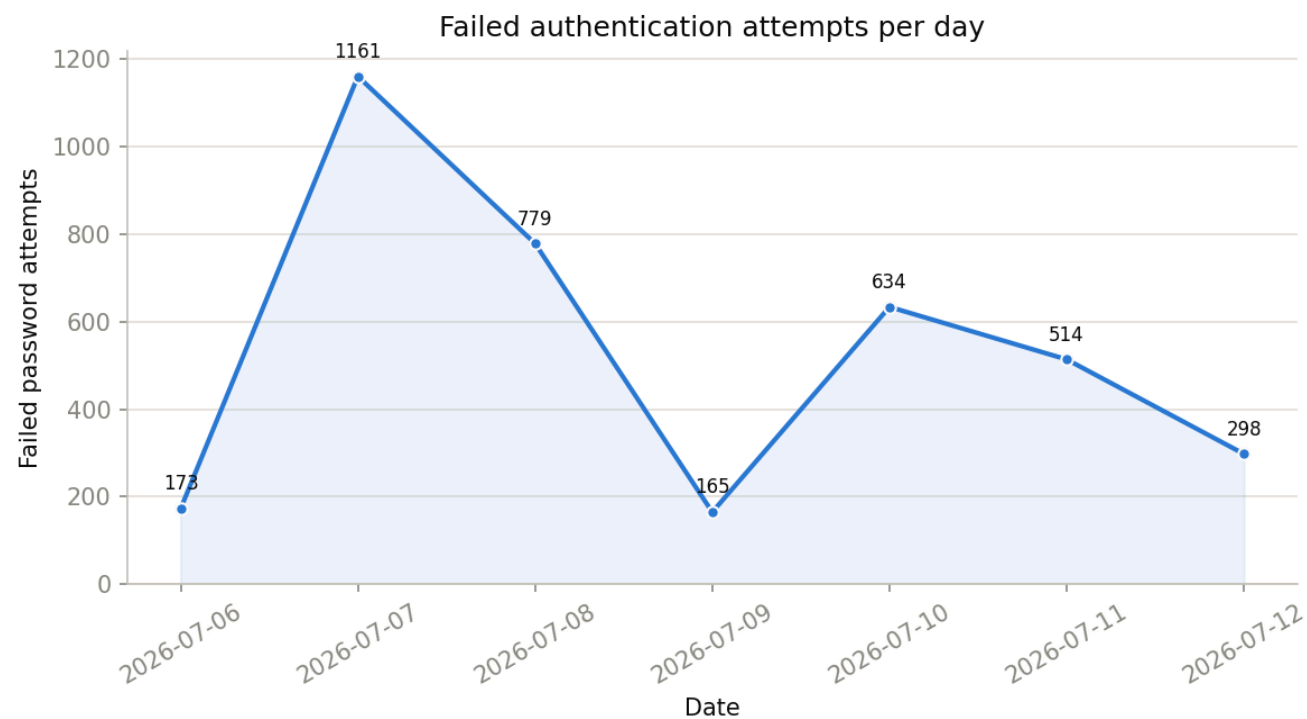


Figure 3. Failed authentication attempts per day, supporting the burst-pattern discussion in Part 1, section 2.

References

- Arkko, J., Cotton, M., & Vegoda, L. (2010). *IPv4 Address Blocks Reserved for Documentation*. <https://doi.org/10.17487/rfc5737>
- fail2ban/fail2ban: Daemon to ban hosts that cause multiple authentication errors. (2024, April 25). GitHub. <https://github.com/fail2ban/fail2ban>
- sshd_config(5) - OpenBSD manual pages. (2018). Openbsd.Org. https://man.openbsd.org/sshd_config
- Rainer Gerhards and Others. (2026). omfwd: syslog Forwarding Output Module. Rsyslog.Com. <https://docs.rsyslog.com/doc/configuration/modules/omfwd.html>
- Grassi, P. A., Fenton, J. L., Newton, E. M., Perlner, R. A., Regenscheid, A. R., Burr, W. E., Richer, J. P., Lefkovitz, N. B., Danker, J. M., Choong, Y.-Y., Greene, K. K., & Theofanos, M. F. (2017). Digital identity guidelines: authentication and lifecycle management. *NIST Special Publication 800-63B*. <https://doi.org/10.6028/nist.sp.800-63b>